

MLA0303_RL_PROGRAM_7.py

```
"""
Experiment 7: Dynamic Programming (Policy Iteration) for an
autonomous taxi routing system to obtain the optimal driving policy.
"""

import numpy as np

GRID = 5
DESTINATION = (4, 4)
ACTIONS = ["UP", "DOWN", "LEFT", "RIGHT"]
DELTA = {"UP": (-1, 0), "DOWN": (1, 0), "LEFT": (0, -1), "RIGHT": (0, 1)}
GAMMA = 0.9
STEP_COST = -1
GOAL_REWARD = 20

STATES = [(r, c) for r in range(GRID) for c in range(GRID)]

def next_state(s, a):
    dr, dc = DELTA[a]
    nxt = (s[0] + dr, s[1] + dc)
    if not (0 <= nxt[0] < GRID and 0 <= nxt[1] < GRID):
        nxt = s
    return nxt

def reward(s, a, s2):
    return GOAL_REWARD if s2 == DESTINATION else STEP_COST

def policy_evaluation(policy, V, theta=1e-4):
    while True:
        delta = 0
        for s in STATES:
            if s == DESTINATION:
                continue
            a = policy[s]
            s2 = next_state(s, a)
            new_v = reward(s, a, s2) + GAMMA * V[s2]
            delta = max(delta, abs(new_v - V[s]))
            V[s] = new_v
        if delta < theta:
            break
    return V

def policy_improvement(V):
    policy = {}
    stable = True
    for s in STATES:
        if s == DESTINATION:
            continue
        best_a, best_v = None, float("-inf")
        for a in ACTIONS:
            s2 = next_state(s, a)
            v = reward(s, a, s2) + GAMMA * V[s2]
            if v > best_v:
                best_v, best_a = v, a
        policy[s] = best_a
    return policy

def policy_iteration():
    V = {s: 0.0 for s in STATES}
    policy = {s: np.random.choice(ACTIONS) for s in STATES if s != DESTINATION}
    for i in range(100):
        V = policy_evaluation(policy, V)
        new_policy = policy_improvement(V)
        if new_policy == policy:
            return policy, V, i + 1
        policy = new_policy
    return policy, V, 100

if __name__ == "__main__":
    np.random.seed(1)
    policy, V, iterations = policy_iteration()
    print(f"Policy Iteration converged in {iterations} iterations.\n")

    state = (0, 0)
    path = [state]
    while state != DESTINATION:
        state = next_state(state, policy[state])
        path.append(state)
    print(f"Optimal route from (0,0) to {DESTINATION}:")
    print(" -> ".join(str(p) for p in path))
    print(f"Number of moves: {len(path) - 1}")
```

PROGRAM OUTPUT

Policy Iteration converged in 9 iterations.

Optimal route from (0,0) to (4, 4):

(0, 0) -> (1, 0) -> (2, 0) -> (3, 0) -> (4, 0) -> (4, 1) -> (4, 2) -> (4, 3) -> (4, 4)

Number of moves: 8